

DocChat 第三周答辩作战手册

- 时间：1 天
- 现状：开源 rag-chatbot 已跑通，自研前端（mock）已完成
- 本周目标：前端对接开源后端 + 讲清 RAG 技术点 + 明确改进方向 + PPT + Q1-Q10
- 分工：A + B 开发对接，C + D 做 PPT/话术

一、前端对接开源后端（A + B）

1.1 rag-chatbot 后端 API 接口

rag-chatbot 后端是 FastAPI（端口 8000），前端是 React+Vite（端口 5173）。你们的前端要对接的核心接口：

接口	方法	功能	请求体	返回
/chat	POST	发送问题，获取回答	{ message, session_id, rag_mode }	流式返回回答文本
/upload	POST	上传文档	multipart/form-data 文件	上传状态
/documents	GET	获取已上传文档列表	—	文档列表
/documents/{id}	DELETE	删除文档	—	删除状态

1.2 A 的具体改动

把前端 mock 数据改成真实 API 调用。核心改动点：

1. 聊天发送：mock 的假回复 → 调 POST /chat

typescript

```
// 之前 (mock)
const mockReply = "这是一段假的回答...";
setMessages([...messages, { role: 'assistant', content: mockReply }]);

// 改成 (真实 API)
const response = await fetch('http://localhost:8000/chat', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    message: userQuestion,
    session_id: currentSessionId,
    rag_mode: true
  })
});
// 处理流式响应 (SSE)
const reader = response.body.getReader();
// ... 逐步读取并更新消息
```

2. 文档上传：mock 的假 loading → 调 `POST /upload`

```
typescript

const formData = new FormData();
formData.append('file', selectedFile);
await fetch('http://localhost:8000/upload', {
  method: 'POST',
  body: formData
});
```

3. 文档列表：写死的假文档 → 调 `GET /documents`

4. CORS 问题：如果前端 5173 调后端 8000 报跨域错误，在后端 `main.py` 里加：

```
python

from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_h
```

1.3 B 的任务

- 确保后端稳定运行，API Key 配好
- 提前用 `memory_builder.py` 索引好演示用的文档
- 准备 2 份演示文档：你们自己的答辩 PDF + 一篇公开论文
- 测试 3-5 个演示问题，确保回答质量好
- 录一段 2 分钟兜底演示视频

1.4 对接不上怎么办

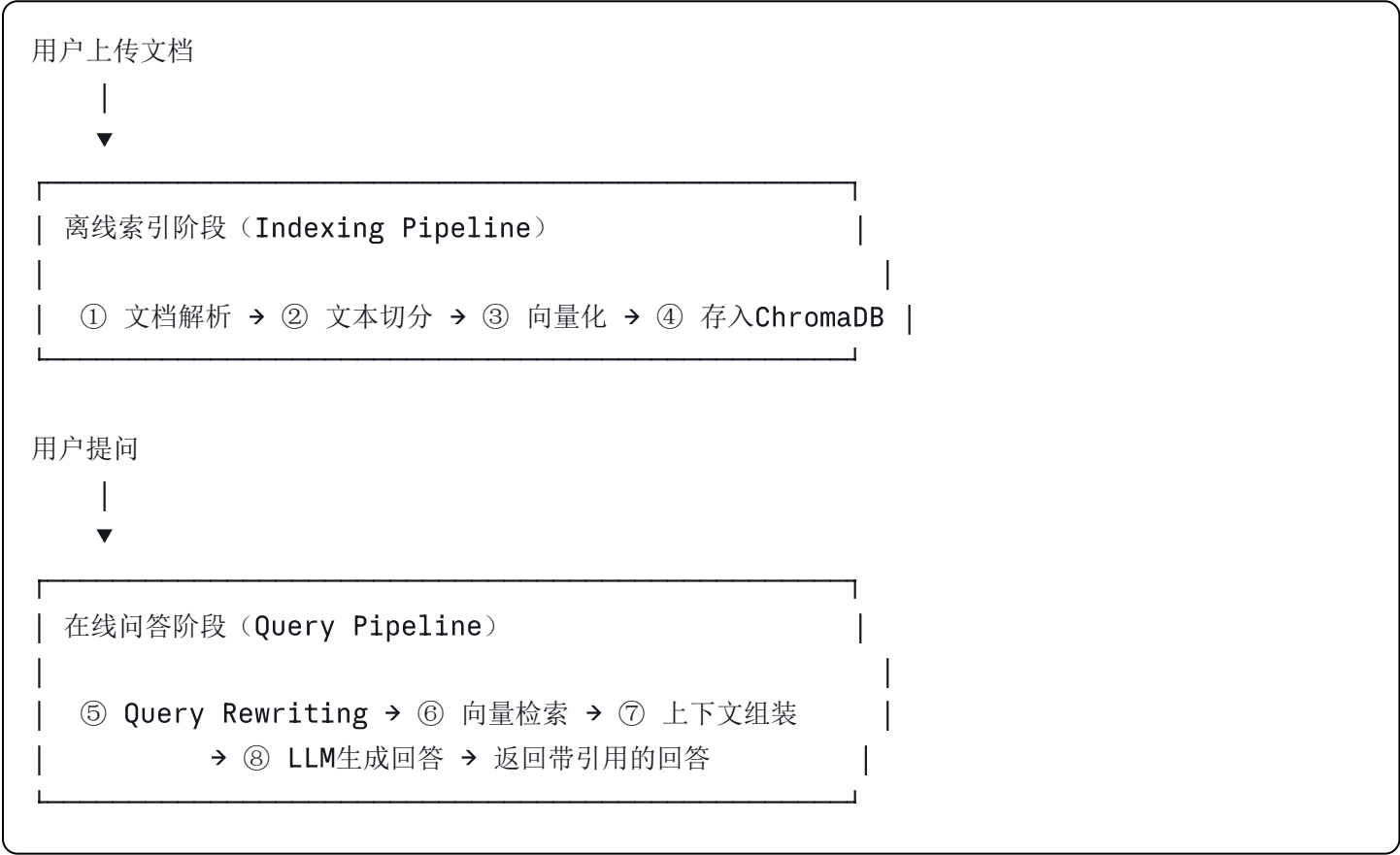
如果 1 天内 API 对接没完全搞定，兜底方案：

- 演示时用 rag-chatbot 自带的 React 前端做功能演示
- 再切到你们自己的前端展示 UI 设计
- 话术："我们的前端正在与后端 API 集成中，目前可以分别演示两部分"

二、RAG 全流程技术点（讲清楚"怎么做的"）

这是答辩核心。你们要能对着系统说清楚每一步在做什么、为什么这么做、有什么可以改进的。

全流程一图总览



① 文档解析（Document Parsing）

当前做法：

- rag-chatbot 用自带的加载器读取 Markdown 文件
- 如果你们扩展了 PDF 支持，用的是 PyMuPDF 提取文本，保留页码元数据

技术要点（向老师讲）：

"文档解析的目标是把不同格式的文件转成纯文本。PDF 解析看似简单，但有很多坑——扫描版 PDF 需要 OCR，多栏排版可能把左右栏文字交错在一起，表格会丢失结构。我们目前优先支持文本型 PDF 和 Markdown，因为这两种格式解析质量最可靠。"

🔧 改进方向：

当前限制	改进方案	难度
只支持 Markdown	加 PDF (PyMuPDF)、Word (python-docx) 支持	★★
无法处理扫描 PDF	集成 OCR (Tesseract 或 PaddleOCR)	★★★
表格解析丢结构	用 camelot 或 tabula 做表格专项提取	★★★
图片中的文字无法提取	多模态模型 (GPT-4o Vision) 识别图片内容	★★★★

② 文本切分 (Chunking)

当前做法：

- 使用从 LangChain 移植的 `RecursiveCharacterTextSplitter`
- 参数：`chunk_size=1000`，`chunk_overlap=50`
- 切分优先级：段落分隔符 `\n\n` → 换行 `\n` → 句号 `.` → 空格 → 强制按字符数切

技术要点 (向老师讲)：

"切分是 RAG 中对最终效果影响最大的一步。我们用 `RecursiveCharacterTextSplitter`，它会按语义层级递归切分——优先在段落边界切，段落太长就在句子边界切。`chunk_size=1000` 字符是经验值，太大会导致检索不精确 (一个 chunk 包含多个主题)，太小会丢失上下文 (一个概念被切断)。`overlap=50` 确保边界信息不丢失。"

🔧 改进方向：

当前限制	改进方案	效果
固定大小切分，不考虑语义	按标题/章节结构切分 (Semantic Chunking)	每个 chunk 语义更聚焦，检索更准
overlap=50 可能不够	调大到 100-200，做 A/B 实验对比	减少边界信息丢失
中文分词不如英文准	用 jieba 分词后再切分	中文场景切分质量提升
chunk_size 是静态值	根据文档类型动态调整 (论文用 500，手册用 1500)	不同场景最优效果不同

③ 向量化 (Embedding)

当前做法：

- 使用 Sentence Transformers 的 `all-MiniLM-L6-v2` 模型 (384 维向量)
- 本地运行，不需要 API 调用
- 每个 chunk 被转换成一个 384 维的浮点数数组

技术要点 (向老师讲)：

"Embedding 的核心思想是：语义相似的文本在向量空间中距离更近。比如 '机器学习' 和 '深度学习' 的向量距离，远小于 '机器学习' 和 '烹饪方法' 的距离。我们用的 `all-MiniLM-L6-v2` 是一个 22M 参数的轻量模型，推理速度快，适合本地部署。"

🔧 改进方向：

当前限制	改进方案	效果
MiniLM 对中文支持一般	换成 <code>m3e-base</code> 或 <code>bge-large-zh</code> (中文专优模型)	中文检索准确率显著提升
384 维精度有限	换成 OpenAI <code>text-embedding-3-small</code> (1536 维)	语义区分度更高
纯文本 embedding	加入标题/元数据做拼接 embedding	chunk 的语义更丰富

④ 向量存储 (Vector Store — ChromaDB)

当前做法：

- 使用 ChromaDB 嵌入式模式 (和后端同进程)
- 索引算法：HNSW (Hierarchical Navigable Small World)
- 存储内容：向量 + 原文 + 元数据 (文件名、页码等)

技术要点 (向老师讲)：

"ChromaDB 底层用 HNSW 索引，这是一种多层图结构的近似最近邻算法。暴力搜索是 $O(n)$ ，HNSW 是 $O(\log n)$ ，在百万级数据上差距是几百倍。但 HNSW 是近似算法，有约 1-5% 的召回损失，对我们的场景完全可接受。"

🔧 改进方向：

当前限制	改进方案	效果
嵌入式部署，数据和应用耦合	ChromaDB 独立服务部署（Client/Server 模式）	数据持久化更安全，可多实例共享
单机存储	迁移到 Milvus / Qdrant（分布式）	支撑百万级向量，水平扩展
无数据隔离	按 user_id 分 collection	多用户数据安全隔离

⑤ 问题重写（Query Rewriting）

当前做法：

- 用户追问时，先用 LLM 结合对话历史将问题改写成独立查询
- 例如：历史="讨论了 RAG 技术" + 追问="它有什么缺点？" → 改写为"RAG 技术有什么缺点？"

技术要点（向老师讲）：

"这一步解决了多轮对话中代词指代的问题。'它有什么缺点'直接拿去检索，'它'没有指代对象，ChromaDB 不知道你问的是什么的缺点。Query Rewriting 通过一次 LLM 调用把问题变成自包含的独立查询，检索准确率在多轮对话场景下提升显著。"

🔧 改进方向：

当前限制	改进方案	效果
每次都要一次 LLM 调用	用规则匹配判断是否需要重写（无代词就跳过）	减少延迟和成本
单一重写策略	HyDE（Hypothetical Document Embedding）：让 LLM 先生成一个假设性回答，用它去检索	检索质量进一步提升
重写质量依赖 LLM	用小模型（gpt-4o-mini）做重写，省成本	重写成本降低 90%

⑥ 向量检索（Retrieval）

当前做法：

- 把重写后的查询做 embedding → 在 ChromaDB 中查找余弦相似度最高的 Top-K 个 chunk
- 默认 K=4

技术要点（向老师讲）：

"检索用的是余弦相似度，它衡量两个向量的方向一致性而非绝对大小，特别适合文本语义比较。K=4 是我们实验后的经验值——K=2 容易漏掉关键信息，K=8 会引入太多噪音稀释上下文质量。"

🔧 改进方向：

当前限制	改进方案	效果
纯向量检索	混合检索：向量检索 + BM25 关键词检索，结果融合	兼顾语义相似和关键词匹配
固定 Top-K	动态 K：根据相似度分数自适应截断	避免引入低相关性 chunk
无重排序	加 Reranker（如 Cohere Rerank 或 bge-reranker）	对 Top-K 结果精排，提升头部精度
召回率未量化	建立评估数据集（标准问答对），计算 Recall@K	有数据支撑优化方向

⑦ 上下文组装 + Prompt Engineering

当前做法：

- 将 Top-K 个 chunk 的原文拼接作为上下文
- 加入系统指令和用户问题，组装成完整 Prompt
- 处理上下文溢出的三种策略（rag-chatbot 特色）：
 - **Create and Refine**：逐个 chunk 顺序处理，每次基于上一次的回答做优化
 - **Hierarchical Summarization**：每个 chunk 独立生成答案，再层级合并
 - **Async Hierarchical Summarization**：并行版层级摘要，大幅提速

技术要点（向老师讲）：

"Prompt 工程有两个关键设计：第一，明确指示 LLM '只基于提供的上下文回答，不要编造'，这是控制幻觉的核心手段；第二，要求 LLM 标注引用来源。当上下文总长度超过 LLM 窗口时，rag-chatbot 实现了三种策略——最有意思的是层级摘要法，它先让 LLM 对每段上下文独立生成答案，再把所有答案合并生成最终回答。"

🔧 改进方向：

当前限制	改进方案	效果
Prompt 模板是静态的	根据问题类型动态选模板（事实型/比较型/总结型）	回答更贴合问题类型
引用标注不够结构化	输出 JSON 结构化引用，前端精确渲染	引用展示更专业
无法处理"文档中无答案"	加入拒答逻辑：相似度低于阈值时直接回复"未找到"	减少胡编乱造

⑧ LLM 生成回答（Generation）

当前做法：

- rag-chatbot 原版用 llama.cpp 跑本地模型（如 Llama 3.1、Phi-3.5）
- 如果你们改成了 API 模式，用的是 OpenAI / DeepSeek API
- 支持流式输出（边生成边显示）

技术要点（向老师讲）：

"我们用流式输出让用户第一时间看到回答开始生成，感知延迟从 5-8 秒降到 1-2 秒。temperature 设为 0 让回答更确定性，避免同一个问题每次回答不一样。"

🔧 改进方向：

当前限制	改进方案	效果
单一模型	简单问题路由到小模型，复杂问题用大模型	降低成本 50%+
无缓存	相同/相似问题做语义缓存	重复问题零延迟
无质量评估	用 RAGAS 框架评估 faithfulness / relevancy	量化回答质量

全流程改进优先级总结

按投入产出比排序，推荐你们答辩时重点提的改进方向：

优先级	改进点	投入	回报	话术关键词
🥇 P0	支持 PDF/Word 多格式	低	高	"扩展用户场景"
🥈 P0	混合检索（向量+BM25）	中	高	"提升召回率"
🥉 P1	中文 Embedding 模型	低	高	"优化中文支持"
🏅 P1	Reranker 精排	中	高	"提升头部精度"
🏆 P2	Semantic Chunking	中	中	"语义完整性"
🏆 P2	语义缓存	中	中	"降低延迟和成本"
🏆 P3	HyDE 假设文档检索	高	高	"前沿检索技术"
🏆 P3	RAGAS 质量评估	高	高	"量化评估体系"

三、PPT 结构（C + D）

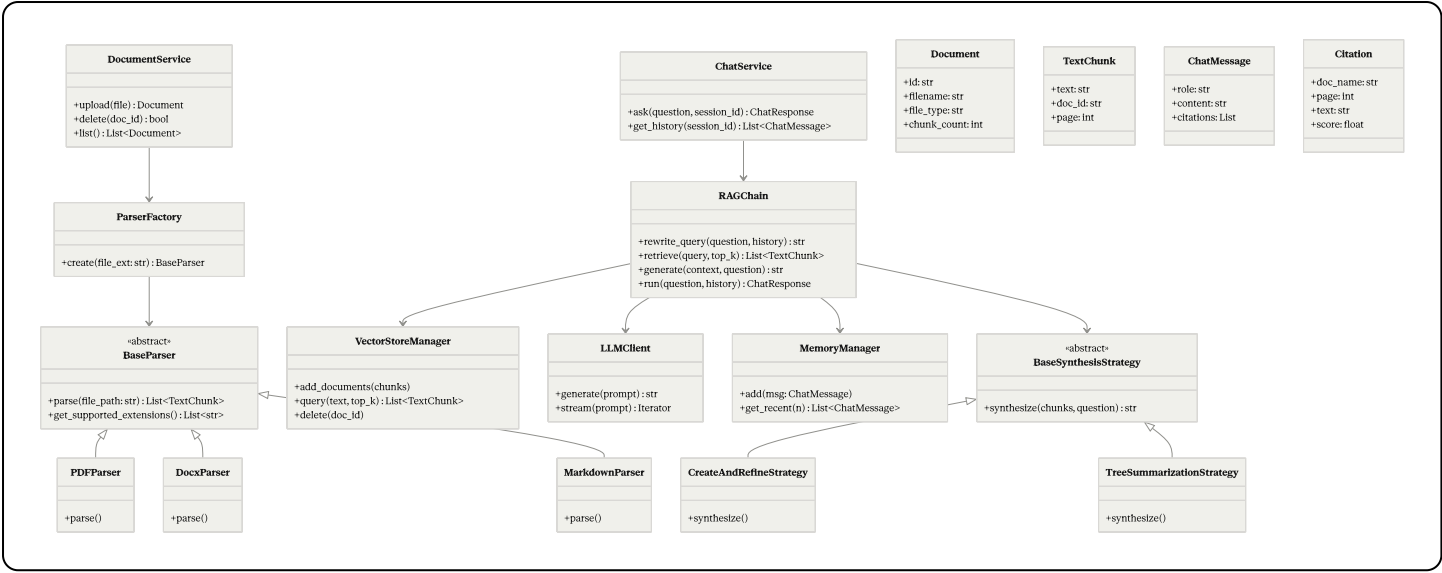
3.1 整体结构（约 15 页）

页码	标题	内容	负责人
P1	封面	DocChat 第三周汇报	C
P2	本周重点	"前后端集成 + RAG 技术实现细节"	C
P3	开发工具（Q1+Q2）	Claude Code + Codex + VS Code	D
P4	技术栈总览（Q3+Q4）	包/组件列表 + 选择理由	D
P5	RAG 流程全景图	八步流程图（本文档第二章的总览图）	C
P6	索引阶段技术点	①解析 ②切分 ③Embedding ④ChromaDB	C
P7	查询阶段技术点	⑤Query Rewriting ⑥检索 ⑦Prompt ⑧生成	C
P8	改进路线图	全流程改进优先级表	C
P9	类图（Q5）	UML 类图	C
P10	对象图（Q6）	运行时实例快照	C
P11	顺序图（Q7）	提问交互时序	C
P12	设计模式（Q8）	策略模式 + 工厂模式 + 单例模式	D
P13	扩展性 + 遗留代码（Q9+Q10）	接口抽象 + 技术债管理	D
P14	前端演示截图 + 演示引导	界面截图	A 提供
P15	进度总结 + Q&A	甘特图对照	C

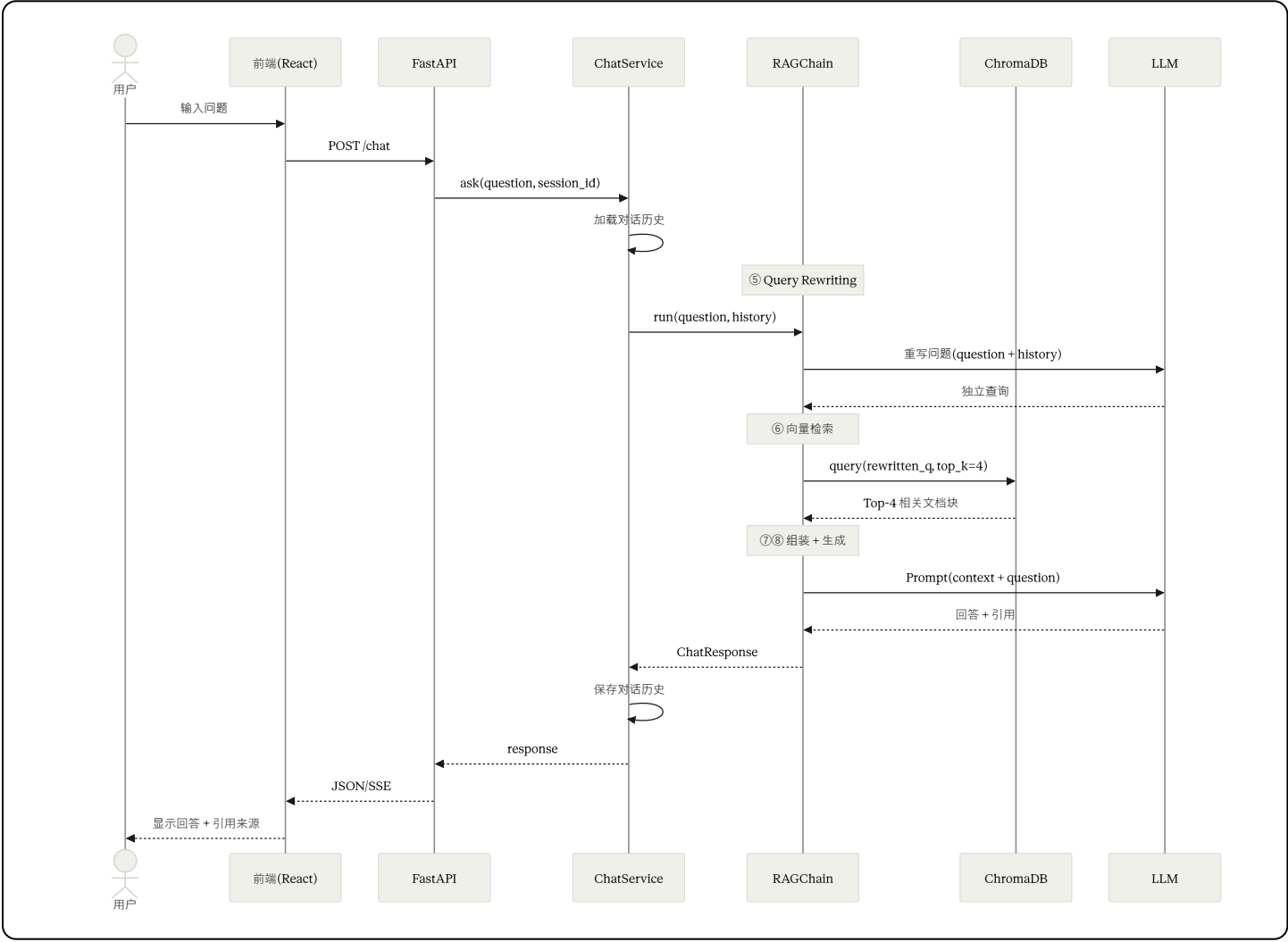
3.2 画 UML 图（C 的核心任务）

工具：mermaid.live（写代码生成图，截图贴 PPT）

类图 Mermaid 代码



顺序图 Mermaid 代码



对象图（文字版，PPT 里用方框画）

场景：用户上传 "课件.pdf" 后提问 "什么是 RAG ?"

```
doc1: Document
  id = "doc_001"
  filename = "课件.pdf"
  file_type = "pdf"
  chunk_count = 42

chunk23: TextChunk
  text = "RAG是检索增强..."
  doc_id = "doc_001"
  page = 5

msg1: ChatMessage
  role = "user"
  content = "什么是RAG?"

msg2: ChatMessage
  role = "assistant"
  content = "RAG是..."
  citations = [cite1]

cite1: Citation
  doc_name = "课件.pdf"
  page = 5
  score = 0.92
```

四、Q1-Q10 话术

Q1：项目中使用了哪种 IDE？

我们主要用 **VS Code** 作为代码编辑器，配合 **Python**、**ESLint**、**Tailwind CSS IntelliSense** 等插件。但更多时候我们在终端环境工作，因为大量使用 **Claude Code** 和 **Codex** 命令行工具来做代码生成和重构。

Q2：是否使用了高级 IDE？

是的。**Claude Code** 是 Anthropic 的命令行 **agentic coding** 工具，能理解整个项目上下文后跨文件完成代码改造。比如我们把参考项目的 LLM 后端从 **llama.cpp** 改成 API 调用，**Claude Code** 自动定位了所有需要修改的位置。**Codex** 是 OpenAI 的 AI 编程助手，我们在对比方案时交替使用。这些工具让编码效率提升了约 2-3 倍。

Q3：是否使用了现有的包或组件？

大量使用。前端用 **React 18 + TailwindCSS + Zustand + Axios**。后端用 **FastAPI + ChromaDB + Sentence Transformers**。文档解析用 **PyMuPDF** 和 **python-docx**。文本切分用从 **LangChain** 移植的 **RecursiveCharacterTextSplitter**。所有依赖在 **pyproject.toml** 和 **package.json** 中管理。

Q4：使用这些框架或组件的好处？

四点：**缩短周期**——FastAPI 自带 API 文档，ChromaDB 零配置启动，节省约 60-70% 代码量。**降低风险**——都是社区验证的成熟方案。**可维护**——文档齐全，新成员可快速上手。**可替换**——向量数据库可换 Milvus，LLM 可换 Claude 或 DeepSeek，因为做了抽象接口。

Q5：类图是什么样的？

（切 PPT）系统分为解析器层、服务层、RAG 核心层和实体层。解析器层用策略模式，BaseParser 定

义抽象接口，PDF/Docx/Markdown Parser 各自实现。RAGChain 组合了 VectorStoreManager、LLMClient、MemoryManager 三个组件。另外还有一个 BaseSynthesisStrategy 抽象类，定义了上下文溢出时的不同处理策略——Create and Refine 和 Tree Summarization 两种实现。

Q6：对象图是什么样的？

（切 PPT）以"用户上传课件.pdf后提问'什么是RAG'"为例：系统中有一个 doc1:Document 实例，被切成了 42 个 chunk。检索命中了 chunk23，是第 5 页关于 RAG 定义的段落，相似度 0.92。用户消息 msg1 和系统回答 msg2 各是一个 ChatMessage 实例，msg2 包含一个 cite1:Citation 指向原文。类图是蓝图，对象图是具体实例的快照。

Q7：顺序图是什么样的？

（切 PPT）用户输入问题 → 前端 POST /chat → ChatService 加载历史 → RAGChain 执行四步：Query Rewriting（一次 LLM 调用改写问题）→ ChromaDB 向量检索 Top-4 → 上下文组装 Prompt → LLM 生成回答。结果逐层返回前端展示。整个流程两次 LLM 调用，重写用小模型控制成本。

Q8：是否使用了设计模式？

策略模式：文档解析器。BaseParser 定义接口，PDFParser/DocxParser/MarkdownParser 各自实现。新增格式只需加一个类。同样，上下文合成策略也用了策略模式——BaseSynthesisStrategy 下面有 Create and Refine 和 Tree Summarization 两种实现。

工厂模式：ParserFactory 根据文件扩展名返回对应解析器实例，调用方无需 if-else。

单例模式：VectorStoreManager 确保 ChromaDB 连接全局唯一，避免重复创建开销。

Q9：如何处理组件变更或功能扩展？

接口抽象：向量数据库、LLM、文档解析器都通过抽象接口通信。换 ChromaDB 为 Milvus 只需实现新的 Manager 类。

插件式新增：新增 Excel 解析只需写 ExcelParser + 在工厂注册。

分层解耦：api → services → rag → storage，依赖单向，任何层的内部重构不影响其他层。

Q10：如何处理遗留代码？

代码审查：GitHub PR + 至少一人 review。**持续重构**：Boy Scout Rule，接触文件时顺手改善。**技术债登记**：GitHub Issues 用 tech-debt 标签跟踪。**AI 辅助**：用 Claude Code 分析代码结构，识别耦合点再给出重构方案。

五、时间表

时段	A（前端）	B（后端）	C（PPT/UML）	D（话术/PPT）
8-10	改前端 mock → 真实 API	确保后端稳定运行 + 准备演示文档	mermaid.live 画类图/顺序图/对象图	写 Q1-Q4, Q8-Q10 话术

时段	A（前端）	B（后端）	C（PPT/UML）	D（话术/PPT）
10-12	调试对接，处理 CORS 等问题	测试演示问答效果	做 PPT P1-P8（流程图+技术点）	做 PPT P3-P4, P12-P13
13-15	练演示流程 3 遍	录兜底视频 + 截图给 C	做 PPT P9-P11（UML 图页）	合并全部话术成文档
15-16	截最终界面截图给 C	确认演示环境就绪	PPT 美化 + 统一风格	话术分发给全员
16-18	全员合练：完整走两遍 PPT → 演示 → Q&A → 修最后问题			

六、演示策略：展示"怎么做的"

演示时不是"看我能用"，而是**每一步都讲技术原理**：

上传文档时说：

"系统现在在做索引——PyMuPDF 提取文本，RecursiveCharacterTextSplitter 按 1000 字切分并保留 50 字重叠，Sentence Transformers 把每段转成 384 维向量，存入 ChromaDB。"

提问时说：

"RAGChain 先做 Query Rewriting 把问题改写成独立查询，然后在 ChromaDB 中用余弦相似度找 Top-4 最相关段落，拼成 Prompt 发给 LLM 生成回答。"

看到回答时说：

"回答中的引用标记对应检索到的原文段落和页码，用户可以点击验证。"

被问到改进方向时说：

"我们计划从三个方面优化：第一，混合检索——结合 BM25 关键词匹配和向量语义检索，提升召回率。第二，加入 Reranker 对检索结果精排。第三，换用中文优化的 Embedding 模型提升中文场景效果。"

七、兜底方案

风险	兜底
前端对接没搞定	用 rag-chatbot 自带 UI 做功能演示 + 切到自研前端展示 UI 设计
现场网络不好	提前录兜底视频

风险	兜底
被追问"代码是你们写的吗"	"我们基于开源参考项目改造，重点是理解了 RAG 全流程每个技术点并明确了改进方向——包括混合检索、中文 Embedding、Semantic Chunking 等"
PPT 来不及	优先级：流程图 > 类图 > 顺序图 > 对象图
被问到不懂的技术细节	诚实说"这是我们下一步要深入研究的改进方向"